

BD Relationnelles

SQL : STRUCTURED QUERY LANGUAGE



- Introduction
- Définition
- Types de bases de données
- Composants d'une BD
- SGBD
- Modélisation des BD
- SQL
 - LDD : Langage de Définition des Données
 - LMD : Langage de Manipulation des Données
 - LID : Langage d'Interrogation des Données
 - LCD : langage de Contrôle des Données



Bases de données

Introduction et définition



- **Définition**

- Une base de données est une collection organisée d'informations structurées, stockées de manière à pouvoir être facilement accédées, gérées et mises à jour. Elle permet de stocker des données de façon systématique afin d'assurer leur cohérence et leur intégrité.

Bases de données

Types de bases de données

- **Bases de données hiérarchiques**
 - Organisées en une structure arborescente.
 - Relations parent-enfant.
 - Exemples : IBM Information Management System (IMS).
- **Bases de données réseau**
 - Modèle plus flexible que le modèle hiérarchique.
 - Les enregistrements peuvent avoir plusieurs relations.
- **Bases de données relationnelles (SGBDR)**
 - Basées sur des tables (relations).
 - Utilisation du langage SQL.
 - Exemples : MySQL, PostgreSQL, Oracle, SQL Server.



Bases de données

Types de bases de données



- **Bases de données orientées objet**
 - Stockage de données sous forme d'objets.
 - Exemples : db4o, ObjectDB.
- **Bases de données NoSQL**
 - Adaptées aux grands volumes de données non structurées.
 - Exemples : MongoDB, Cassandra, Redis.



Composants d'une BD Relationnelle

- **Table**
 - Collection d'enregistrements sous forme de lignes et colonnes.
- **Enregistrement (Ligne)**
 - Ensemble de données associées à un objet unique.
- **Champ (Colonne)**
 - Catégorie d'information dans une table.
- **Clé primaire**
 - Identifiant unique pour chaque enregistrement.
- **Clé étrangère**
 - Lien entre deux tables différentes.



Bases de données

SGDB



- **Système de Gestion de Base de Données**
 - Un SGDB est un logiciel permettant de créer, manipuler et administrer une base de données.
- **Fonctions principales**
 - Stockage et récupération des données.
 - Gestion des transactions.
 - Contrôle de l'accès aux données.
- **Exemples de SGDB**
 - SGDB relationnels : MySQL, PostgreSQL, Oracle.
 - SGDB NoSQL : MongoDB, CouchDB.

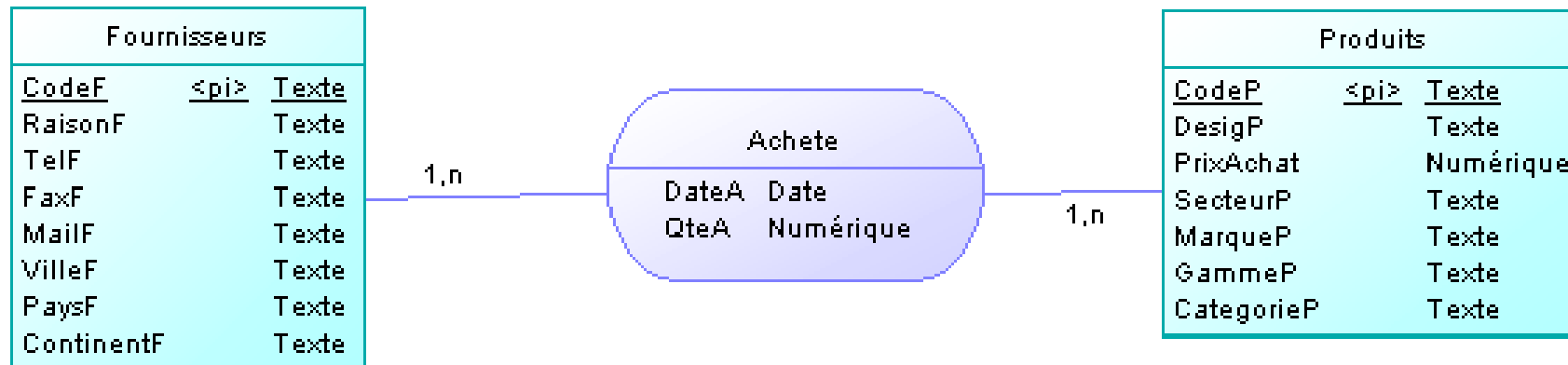


Bases de données

Modélisation des Données



- **Modèle conceptuel (MCD)**
 - Représentation des entités et relations entre elles.
- **Modèle logique (MLD)**
 - Traduction du modèle conceptuel en termes techniques (tables, clés).
- **Modèle physique (MPD)**
 - Implémentation concrète de la base de données.



Langage de Définition des Données : LDD

- Ordre **CREATE**
 - Création de la structure de l'objet (Table, Index, ...)
- Ordre **DROP**
 - Suppression des données et de la structure
- Ordre **ALTER**
 - Modification de la structure (contenant)

LDD : Create

```
CREATE TABLE nom_table (  
    colonne1 type1(longueur1),  
    colonne2 type2(longueur2),  
    ...  
    CONSTRAINT nom_contrainte1 type_contrainte1,  
    ...  
);
```

- types de contraintes :
 - **PRIMARY KEY**
 - **FOREIGN KEY**
 - **CHECK**
 - **NOT NULL**
 - **UNIQUE**



Bases de données

LDD : Les types de données



VARCHAR2(size)	Chaîne de caractères de longueur variable
CHAR(size)	Chaîne de caractères de longueur fixe
NUMBER(p,s)	Numérique de longueur variable
DATE	Valeurs de date et d'heure
LONG	Données caractères de longueur variable (2 Go)



LDD : Drop

- Suppression complète de la table : définition et données

```
Drop table nom_table;
```

- Suppression aussi des contraintes de référence filles

```
Drop table nom_table cascade constraints;
```

Exemples :

```
Drop table Client cascade constraints;
```

```
Drop table commande;
```

LDD : Alter

- Ajout de colonnes

```
ALTER TABLE nom_table  
ADD (col1 type1, col2 type2);
```

- Modification de colonnes

```
ALTER TABLE nom_table  
MODIFY (col1 type1, col2 type2);
```

- Suppression de colonnes

```
ALTER TABLE nom_table  
DROP COLUMN (col1, col2);
```



Alter : Exemples

```
Alter table Client add (  
Fax Varchar2(15) );
```

```
Alter table Client Modify (  
NomClt Varchar2(40) );
```

```
Alter table Client drop column Fax;
```

Alter : Modification des contraintes

- Ajout de contraintes

```
ALTER TABLE nom_table  
ADD CONSTRAINT nom_cont type_cont;
```

Alter table Client **add(constraint** ch_numclt check(NumCl between 1 and 9999));

- Suppression de contraintes

```
ALTER TABLE nom_table  
DROP CONSTRAINT nom_contrainte;
```

Alter table Client **drop constraint** ch_numclt ;



Alter : Activation et désactivation de contraintes

- Activation et Désactivation de contraintes

```
ALTER TABLE nom_table  
ENABLE | DISABLE CONSTRAINT nom_cont;
```

```
ALTER TABLE nom_table  
ENABLE | DISABLE constraint primary key;
```

Objet séquence : générateur de nombres

- Permet d'obtenir des valeurs incrémentales
- Verrouillage automatique en cas de concurrence d'accès
- Valeur suivante : `nom_séquence.NEXTVAL`
- Valeur courante : `nom_séquence.CURRVAL`

```
CREATE SEQUENCE nom_séquence  
START WITH valeur_départ  
INCREMENT BY incrément;
```

```
INSERT INTO Table1 VALUES  
(nom_séquence.NEXTVAL, ...);
```

```
DROP SEQUENCE nom_séquence;
```



Bases de données

Objet séquence : Exemples



```
SQL> CREATE TABLE Exemple (Numero number, Chaine varchar2(25));
```

```
SQL> CREATE SEQUENCE seq_Exemple START WITH 10 INCREMENT BY 2;
```

Séquence créée.

```
SQL> INSERT INTO Exemple VALUES(seq_Exemple.NEXTVAL, 'Ahmed');
```

```
SQL> INSERT INTO Exemple VALUES(seq_Exemple.NEXTVAL, 'Sara');
```



Bases de données

Objet séquence : Exemples



```
SQL> select * from Exemple;
```

```
NUMERO CHAINE
```

```
-----
```

```
10 Ahmed
```

```
12 Sara
```

```
SQL> SELECT seq_Exemple.CURRVAL FROM dual;
```

```
CURRVAL
```

```
-----
```

```
12
```

- **INSERT**
 - Ajout de ligne(s) dans une table
- **UPDATE**
 - Mise à jour (modification) de une ou plusieurs colonnes de une ou plusieurs lignes
- **DELETE**
 - Suppression de une ou plusieurs lignes
- **COMMIT / ROLLBACK**
 - Fin d'une transaction

LMD : INSERT

INSERT INTO nom_table [(liste des colonnes)] (liste des valeurs);

Exemples :

```
INSERT INTO client values (2, 'Mahmoud', 'Casa');
```

```
INSERT INTO client (numclt, nomclt, ville) values (3, 'Ilham', 'Casa');
```

```
INSERT INTO client select * from Table1 where ...;
```



```
UPDATE nom_table  
SET col1 = val1 [,col2 = val2 ...]  
[ WHERE prédicat];
```

Exemples :

```
update client set nomclt='Hatim' where numclt= 2;
```

```
update client set nomclt='Hatim', ville='Casa' where numclt= 2;
```



```
DELETE FROM nom_table  
[WHERE prédicat];
```

Exemples :

```
delete from client where numclt=2;
```

```
delete from client;
```




Bases de données

LMD : COMMIT / ROLLBACK



- Notion de **transaction** : ensemble fini d'actions (update, delete et insert)
- **Commit** : point de confirmation dans la base
- **Rollback** :(Annulation)



- Ordre SELECT
- Sélection : restriction sur les lignes
- Projection : fonctions de groupage et restrictions sur les groupages
- Ordres de Tris
- Fonctions Chaînes et Calculs
- Mise en page des résultats

Ordre SELECT

```
SELECT {DISTINCT|*|col1[,col2,...]}  
  [AS nom_col]}  
FROM nom_de_table  
WHERE <prédicat sur les lignes>  
GROUP BY col1 [,col2,...]  
HAVING <prédicat sur les groupages>  
ORDER BY {col1{ASC|[,col2 ... ]|n° col }
```

Select : Exemples

```
SELECT *
```

```
FROM Clients;
```

← Toutes les colonnes

```
SELECT NomClt AS "Nom du client"
```

```
FROM Clients;
```

↑ Alias

```
SELECT NumClt, NomClt
```

```
FROM Clients
```

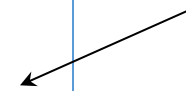
```
WHERE Lower(ville) LIKE 'casa';
```

↑ Prédicat de restriction

Select : Clause ORDER BY, DISTINCT

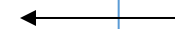
```
SELECT *  
FROM commandes  
ORDER BY datecde DESC ;
```

Trie par ordre
décroissant

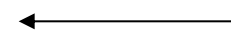


```
SELECT Numero, ROWNUM  
FROM commandes  
WHERE ROWNUM <= 5;
```

Colonne virtuelle

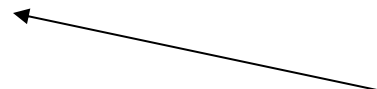


Affichage des 5 premiers



```
SELECT DISTINCT DateCde  
FROM commandes;
```

Élimine des doublons



Select : Group by

- Clause de groupage : GROUP BY

```
SELECT dateCde, Count(*)  
FROM Commandes  
GROUP BY dateCde;
```

- Restriction sur les groupages : HAVING

```
Select dateCde, COUNT(*) as nbreCdes  
From commandes  
GROUP BY dateCde  
HAVING COUNT(*)>10;
```

Fonctions chaînes de caractères

CHR	Retourne le caractère ASCII équivalent
INITCAP	Premières lettres de chaque majuscule
LOWER	Tout en minuscules
UPPER	Tout en majuscules
TO_CHAR(entier)	Transforme en chaîne
TO_NUMBER(chaine)	Transforme en entier

Bases de données

Fonctions pour les dates

ADD_MONTHS(<i>d</i> , <i>n</i>)	Ajoute <i>n</i> mois à la date <i>d</i>
LAST_DAY(<i>d</i>)	Retourne le dernier jour du mois
MONTHS_BETWEEN(<i>d1</i> , <i>d2</i>)	Retourne le nombre de mois entre 2 dates
NEXT_DAY(<i>d</i> , <i>chaîne</i>)	Retourne le prochain jour ouvrable
SYSDATE	Date courante
TO_DATE(<i>chaîne</i> , <i>format</i>)	Exemple : 'DD-MM-YYYY'



Ordres d'entrée – sortie : variables paramétrées

ACCEPT variable [**NUMBER** | **CHAR**]
[**PROMPT** [texte] | **NOPROMPT**] [**HIDE**]

SQL> ACCEPT V_Cde Char **PROMPT** 'Taper le numéro de la commande : '

Taper le numéro de la commande : 10

Select * From Commandes
Where Numero='&**V_Cde**';

Bases de données

LID - Multi-Table : Jointures

- Opérateurs ensemblistes
 - **UNION, INTERSECT et MINUS**
- Jointure en forme procédurale déconnectée
 - **SELECT imbriqués déconnectés**
- Jointure en forme procédurale synchronisée
 - **SELECT imbriqués synchronisés**
- Jointure en forme non procédurale ou relationnelle
 - Pas d'imbrication : 1 seul bloc SELECT-FROM-WHERE
 - Jointure complète ou externe (gauche ou droite)
- Requêtes hiérarchiques
 - Parcours d'arbre : association réflexive 1-N



```
SELECT colonnes résultats (des 3 tables)
FROM table1 t1, table2 t2, table3 t3
WHERE t1.col1=t2.col2 AND t2.col2=t3.col3;
```

- Un seul bloc Select ... From ... Where
- Présentation d'une jointure naturelle
- N tables → N-1 Jointures

Bases de données

Opérateurs ensemblistes

Opérateur	Résultat
UNION*	Renvoie toutes les lignes sélectionnées par les deux requêtes en excluant les lignes identiques
INTERSECT*	Combine deux requêtes et retourne uniquement les valeurs du premier Select qui sont identique à au moins une de celles du second Select
MINUS*	Renvoie toutes les lignes retournées par le premier Select qui ne sont pas retournées par le second Select

* Même nombre de colonnes et mêmes types de colonnes

Syntaxe :

```
SELECT liste de colonnes FROM table1  
UNION | MINUS | INTERSECT  
SELECT liste de colonnes FROM table2;
```

Composition des opérateurs ensemblistes :

```
SELECT liste de colonnes FROM table1  
INTERSECT  
(SELECT liste de colonnes FROM table2  
MINUS  
SELECT liste de colonnes FROM table3);
```

Jointure forme procédurale déconnectée : SELECT imbriqués

```
SELECT liste de colonnes(*) FROM table1  
col1 [NOT] [IN] [<,>,<!=,>=,...][ANY | ALL]  
(SELECT col2 FROM table2  
[WHERE prédicat ]);
```

- Col1 et col2 sont les colonnes de jointure
- Ne sélectionne que des colonnes de la première table (*)
- C'est la sous-requête de niveau le plus bas qui est évaluée en premier, puis la requête de niveau immédiatement supérieur, ...

- Sous-requêtes monolignes : <, >, !=, =, >=, <=
- Sous-requêtes multilignes : IN, ANY et ALL



Sous-requêtes multilignes : IN, ANY et ALL

- IN** Compare un élément à une donnée quelconque d'une liste ramenée par la sous-requête.
- ANY** Au moins un élément
- « =ANY » équivaut à IN
 - « <ANY » inférieur à au moins une des valeurs
 - « >ANY » supérieur à au moins une des valeurs
- ALL** Compare l'élément à tous ceux ramenés par la sous-requête.
- « <ALL » inférieur au minimum
 - « >ALL » supérieur au maximum

Jointure en forme procédurale synchronisée

```
SELECT T1.Colonne FROM table T1 WHERE .....  
      (SELECT T2.Colonne FROM table T2  
      WHERE T2.Col2=T1.Col1);
```

- Règles :**
- le prédicat de la sous-interrogation fait référence à une colonne de l'interrogation principale
 - si une table est présente dans les 2 select, la renommer

Exécution :

- L'exécution de la sous-requête se fait pour chaque ligne de l'interrogation principale.



Jointure en forme procédurale synchronisée

Une sous-requête synchronisée peut ramener une ou plusieurs lignes. Différents opérateurs peuvent être employés (=, >, <, >=, <=, **EXISTS**)

EXISTS permet d'interrompre la sous-requête dès le premier enregistrement trouvé.

- Le prédicat 'EXISTS' est « True » si le sous-select ramène au moins une ligne
- NOT EXISTS retourne « True » si aucun enregistrement n'est extrait par le sous-select



Bases de données

L'objet VUE



- Une VUE est une table virtuelle : aucune implémentation physique de ses données
- La définition de la vue est enregistrée dans le DD
- A chaque appel d'une vue : le SGBD réactive sa construction à partir du DD
- Vue mono-table : crée à partir d'une table
 - Modifications possibles → modifications dans la table
 - Jointure en forme procédurale autorisée
- Vue multi-table
 - Créée par une jointure en forme relationnelle
 - Aucune modification autorisée

Création et suppression d'une VUE

- Création d'une vue

```
CREATE [( OR REPLACE )] VIEW nom_vue  
liste des colonnes de la vue )]  
AS  
SELECT .....  
[WITH CHECK OPTION [CONSTRAINT nom_cont]];  
Sélection de lignes et colonnes
```

- Suppression d'une vue

```
DROP VIEW nom_vue;
```



Bases de données

Vues Mono-table



- Vue mono-table avec restriction horizontale
- Vue mono-table avec restriction verticale
- Vue mono-table avec colonnes virtuelles
 - Pas de modification sur les colonnes virtuelles
- Modifications autorisées sur les colonnes de base → mise à jour instantanée !
- Vue mono-table avec groupage



Bases de données

Vues multi-tables



- Simplification de requêtes
- Pas de modifications possibles dans ce type de vue (voir trigger « instead of »)
- Tables temporaires « virtuelles » de travail
- Transformation de la présentation des données → Schéma externe

Vues avec Contraintes WITH CHECK OPTION

- Principe : le prédicat de sélection de lignes se transforme en contrainte
- Mise en place de contraintes spécifiques

La division

- La division permet de trouver les éléments d'un ensemble qui sont associés à tous les éléments d'un autre ensemble.

Soit la BD suivantes :

- ETUDIANT(id_etudiant, nom)
- LIVRE(id_livre, titre)
- EMPRUNT(id_etudiant, id_livre)

La division inexacte

Table ETUDIANT

id_etudiant	nom
1	Ahmed
2	Zineb
3	Kamal
10	Leila

Table LIVRE

id_livre	titre
101	SQL pour débutants
102	Apprendre Python
103	Data Science Intro

Table EMPRUNT

id_etudiant	id_livre
1	101
1	102
2	101
2	102
2	103
3	101
10	101
10	102

- Donner la liste des étudiants qui ont emprunté **tous les livres** empruntés par l'étudiant n°10.

```
SELECT distinct E1.id_etudiant
FROM ETUDIANT E1
WHERE NOT EXISTS
(
    SELECT id_livre FROM EMPRUNT E2 WHERE E2.id_etudiant = 10
    MINUS
    SELECT id_livre FROM EMPRUNT WHERE id_etudiant = E1.id_etudiant
)
AND E1.id_etudiant != 10;
```

La division exacte

Table ETUDIANT

id_etudiant	nom
1	Ahmed
2	Zineb
3	Kamal
10	Leila

Table LIVRE

id_livre	titre
101	SQL pour débutants
102	Apprendre Python
103	Data Science Intro

Table EMPRUNT

id_etudiant	id_livre
1	101
1	102
2	101
2	102
2	103
3	101
10	101
10	102

Donner la liste des étudiants qui ont **exactement les mêmes livres empruntés** que l'étudiant n°10.

```
SELECT distinct E1.id_etudiant FROM ETUDIANT E1
WHERE NOT EXISTS (
    SELECT id_livre FROM EMPRUNT E2 WHERE E2.id_etudiant = 10
    MINUS
    SELECT id_livre FROM EMPRUNT WHERE id_etudiant = E1.id_etudiant)
AND NOT EXISTS (
    SELECT id_livre FROM EMPRUNT WHERE id_etudiant = E1.id_etudiant
    MINUS
    SELECT id_livre FROM EMPRUNT E2 WHERE E2.id_etudiant = 10 )
AND E1.id_etudiant != 10;
```


Exercice

Table ETUDIANT

id_etudiant	nom
1	Ahmed
2	Zineb
3	Kamal
10	Leila

Table LIVRE

id_livre	titre
101	SQL pour débutants
102	Apprendre Python
103	Data Science Intro

Table EMPRUNT

id_etudiant	id_livre
1	101
1	102
2	101
2	102
2	103
3	101
10	101
10	102

1. Nombre de livres empruntés par chaque étudiant
2. Étudiant ayant emprunté le plus de livres
3. Livres jamais empruntés
4. Nombre total de livres différents empruntés
5. Afficher les noms des étudiants ayant emprunté le livre « Python 10 »
6. Donner la liste des livres empruntés par au moins 2 étudiants différents
7. Pour chaque livre, afficher le nombre d'étudiants qui l'ont emprunté
8. Étudiants ayant emprunté plus de la moitié des livres disponibles
9. Afficher les étudiants qui ont emprunté exactement 2 livres
10. Donner les paires d'étudiants ayant emprunté exactement les mêmes livres